NOVA IMS
INFORMATION MANAGEMENT SCHOOL

Master's Degree in
Data Science and Advanced Analytics

Specialization in
Business Analytics

Identifying Ponzi-like tokens through Blockchain Transaction Networks on Ethereum

Alexandre José Ribeira Alves

NOVA Information Management School
Instituto Superior de Estatística e Gestão de Informação
Universidade NOVA de Lisboa

NOVA Information Management School
Instituto Superior de Estatística e Gestão de Informação
Universidade Nova de Lisboa

Identifying Ponzi-like tokens through Blockchain Transaction Networks on Ethereum

by

Alexandre José Ribeira Alves

Master Thesis presented as partial requirement for obtaining the Master's degree in Data Science and Advanced Analytics, with a specialization in Business Analytics

Supervised by

Flávio L. Pinheiro, NOVA Information Management School

December, 2025

STATEMENT OF INTEGRITY

I hereby declare having conducted this academic work with integrity. I confirm that I have not used plagiarism, any form of undue use of information or falsification of results along the process leading to its elaboration. I further declare that I have fully acknowledged the Rules of Conduct and Code of Honor from the NOVA Information Management School.

27th of December 2025

Alexandre Alves

ACKNOWLEDGEMENTS

I would like to express my gratitude to my supervisor, Professor Flávio Pinheiro, for his guidance, availability, and direct feedback throughout this project.

I would also like to thank my parents and brothers for their continued support, and my girlfriend Beatriz, whose patience and encouragement made this process considerably easier.

ABSTRACT

This thesis investigates whether Smart Ponzi schemes on Ethereum can be identified solely from transaction graph features, without access to contract source code or bytecode. The central hypothesis is that Smart Ponzi schemes exhibit behavioural and temporal patterns that can be used to distinguish them from normal Dapps. Transaction data was collected from Google BigQuery's public Ethereum dataset, using a ground truth dataset retrieved from the literature as the labels. The ground truth dataset comprised 1539 contracts, comprising 311 Ponzi schemes and 1228 normal Dapps. After processing the dataset to remove contracts with fewer than five unique senders, the final dataset comprised 1190 contracts. Four features were used in the final models: *in_out_ratio, out_degree, new_sender_decay, and in_degree*. These features were used to train four models (Logistic Regression, Random Forest, XGBoost, SVM). Two models emerged as the best options, with Random Forest and XGBoost achieving AUC scores of 0.925 and 0.926, and macro F1 scores of 0.814 and 0.816, respectively. A sensitivity analysis was performed to check whether contracts were being classified based on contract scale. This analysis confirmed that temporal and behavioural features carried genuine discriminative power. An ablation study confirmed that the features used in the main models contributed independent discriminative power, with in_out_ratio taking the central role. The dominant role of this feature is partially explained by a data collection limitation, the lack of ERC-20 token transfers. Despite this limitation, the results show that transaction-based behavioural features can produce competitive Ponzi detection models without requiring code access, offering broader applicability when compared to methods that require source code or bytecode access.

KEYWORDS

Ethereum; Smart Contracts; Ponzi Scheme Detection; Transaction Graph Analysis; Machine Learning; Blockchain Fraud



NOVA Information Management School
Instituto Superior de Estatística e Gestão de Informação
Universidade NOVA de Lisboa

TABLE OF CONTENTS

1 Introduction....9
2 Literature review....11
2.1 Dataset Construction and Feature Engineering....11
2.2 Model Performance and Research Gap....14
3 Methodology....16
3.1 Data....16
3.2 Transaction Graph Construction....17
3.3 Feature Engineering....18
3.4 Data Cleaning and Splitting....20
3.5 Feature Selection....20
3.6 Classification Models....21
4 Results and Discussion....23
4.1 Exploratory Data Analysis....23
4.2 Classification Models....27
5 Conclusions and Future Work....34
Bibliographical References....36
Appendix A....39

LIST OF FIGURES

Figure 1 - Value Distribution Inequality by Contract Size 24
Figure 2 - Degree Distribution: Ponzi VS Normal DApp 25
Figure 3 - In-Out Ratio Distribution: Ponzi VS Normal DApp 25
Figure 4 - Temporal growth of Unique senders: Ponzi VS Normal DApp 26
Figure 5 - Feature Distributions: Ponzi VS Normal Dapp 27
Figure 6 - Confusion Matrix: Random Forest 28
Figure 7 - Confusion Matrix: XGBoost 29
Figure 8 - ROC curve comparison 30
Figure 9 - Feature Importance: Random Forest vs XGBoost 31
Figure 10 - Ablation Study 31

LIST OF TABLES

Table 1 - Data Dimensions and Values 17
Table 2 - Results from Feature Engineering 19
Table 3 - Results from Feature Selection 21
Table 4 - Classification Results for Ponzi Label 28
Table 5 - Model comparison with other relevant studies 32
Table 6 - Model Comparison with SMOTE 39

LIST OF ABBREVIATIONS AND ACRONYMS

DApp	Decentralised Application
ERC-20	Ethereum Request for Comments 20 (token standard)
EVM	Ethereum Virtual Machine
GCN	Graph Convolutional Network
GAT	Graph Attention Network
SMOTE	Synthetic Minority Over-sampling Technique
AUC	Area Under the Curve
SVM	Support Vector Machine
CRBG	Contract Runtime Behaviour Graph



NOVA Information Management School
Instituto Superior de Estatística e Gestão de Informação
Universidade NOVA de Lisboa

1 INTRODUCTION

Blockchain technology promised a financial system free of intermediaries, one that would be open, transparent, and governed by code rather than institutions. Ethereum's advent fulfilled this promise more fully than Bitcoin's. With the introduction of Ethereum smart contracts, these contracts became self-executing programs that operate without human oversight on a public ledger (Buterin, 2014). Yet the properties that made Ethereum such an interesting prospect have also made it an environment where cybercrime can flourish. Permissiveness of deployment, immutability and pseudo-anonymity have made Ethereum a perfect vehicle for fraud.

Financial crime remains a significant issue for cryptocurrencies. From 2021 to 2022, the US Federal Trade Commission reported that over 46000 people lost more than one billion dollars to cryptocurrency scams (Krishnan et al., 2023; *Reports Show Scammers Cashing in on Crypto Craze*, 2022). Research focusing specifically on Ponzi schemes has identified over 1,500 Ponzi schemes related to Bitcoin (Vasek & Moore, 2019) revealing the scale of the issue.

The American Securities Exchange Commission defines Ponzi schemes as an investment fraud that pays existing investors with funds collected from new investors (*Ponzi Scheme | Investor.Gov*, n.d.). This definition closely aligns with the on-chain behaviour of these contracts. Victims face consequences similar to those of traditional crypto-agnostic schemes, which, by design, collapse as recruiting new prospects becomes more difficult (Artzrouni, 2009; Peng & Boyle, 2025).

Detection remains a challenging task. The most effective methods, such as Jung et al.'s 0-day classification, depend on access to contract bytecode or Solidity source code (Jung et al., 2019). This dependence represents a significant structural limitation due to the reduced availability of public source code. Methods relying on these features can only be applied to the small subset of contracts that make this code publicly available. Therefore, a methodology that can work across all contracts must not depend on these features.

Transaction data presents a compelling alternative. One of Ethereum's characteristics is that contract inflows and outflows are permanently recorded on a public ledger, regardless of whether the source code is available. Earlier studies, such as Hu et al. (2021), have shown that time-series features derived from transaction sequences can identify Smart Ponzi contracts without relying on code-level data. Yu et al. (2021) approached the problem as a node classification task on transaction networks and discovered that combining the network structure with basic transaction features enhances model performance. These findings indicate that behavioural patterns within transaction networks can help distinguish Smart Ponzi schemes from legitimate DApps.

This thesis aims to investigate that possibility. The central hypothesis is that Smart Ponzi schemes exhibit structural patterns in their inflow and outflow transaction networks that

distinguish them from legitimate decentralised applications (DApps). The approach is exclusively based on transactions and aims to use these behavioural patterns, turning them into features for supervised machine learning classifiers. Four features were constructed from the aggregated transaction networks of 1539 labelled contracts. After preprocessing, the dataset contained 1,190 contracts, including 167 Smart Ponzi Schemes and 1,023 standard DApps. These features were employed to train and evaluate four models: Logistic Regression, Random Forest, XGBoost, and Support Vector Machine.

The thesis is organised as follows. Chapter 2 lays the theoretical foundation, providing an exhaustive exploration of existing methods, datasets, and feature types. Chapter 3 describes the project's methodology, including data collection, feature engineering, feature selection and the classification model pipeline. Chapter 4 presents the results and the exploratory data analysis. This section evaluates the models' performance and addresses questions that arose during model development. Finally, chapter 5 provides conclusions, discusses limitations and suggests directions for future research.

2 LITERATURE REVIEW

The US Securities and Exchange Commission defines Ponzi schemes as a type of investment fraud in which returns paid to existing investors are funded by recruiting new investors (*Ponzi Scheme | Investor.Gov*, n.d.). This structural requirement for new recruits means that every Ponzi scheme inevitably collapses.

This type of fraud predates blockchain technology by over a century, having been pioneered by Charles Ponzi with his arbitrage of reply coupons . The advent of cryptocurrency has created a new environment in which it can thrive. Bitcoin's pseudo-anonymity, lack of regulatory oversight, and global accessibility have made it an attractive vehicle for fraudsters. Ethereum would later deepen this issue with the addition of smart contracts, which allow the automation of fraud by making transactions immutable and self-executing (Buterin, 2014; Wood, 2014).

With these new technologies, these frauds became easily scalable, with reports of thousands of victims and more than $17 million US dollars lost to fraudulent contracts (Chen et al., 2021). These figures almost certainly underestimate the real scope of the issue, since they reflect only the schemes that had been caught up to the time of the research.

This credible threat motivates research into automated detection. Presently, the literature has developed along two methodological avenues: approaches that analyse contract code to identify signatures of fraudulent logic, and approaches that analyse transaction records to identify behavioural patterns consistent with Ponzi dynamics.

2.1 Dataset Construction and Feature Engineering

One of the recurring issues in the literature is the absence of a gold-standard ground-truth dataset. Obviously, Ponzi schemes do not self-identify, so every study must first make one of two choices: either construct a new labelled dataset or reuse a previously used dataset.

Early work relied on the manual inspection of the Solidity code of open-source contracts. This research identified 184 Ponzi contracts and made them publicly available (Bartoletti et al., 2020). The labelled dataset was then utilised by other researchers, such as Chen et al. (2018), who rechecked the contracts, resulting in 131 Ponzi Contracts and 1251 non-Ponzi contracts (Chen et al., 2018). Liang et al. (2024) also utilised this contract again, rechecking the contracts. This time, the researchers excluded 33 contracts without source code, resulting in a final ground truth set with 139 Ponzi contracts and 1300 non-Ponzi contracts (Liang et al., 2024). Other authors extended the dataset. Chen et al. (2019) included 200 Ponzi Schemes and 3,580 non-Ponzi contracts. This same dataset would be used by Hu et al. (2021) (Chen et al., 2019; Hu et al., 2021). Zhang et al. (2022) would reuse this dataset, keeping the same labels but applying them to newly collected transaction, opcode, and bytecode data (Zhang et al., 2022).

Chen et al. (2021) used the dataset made available by Bartoletti et al., but used symbolic execution (a static analysis technique that explores all possible execution paths of a program) to formally verify each contract's execution path. They audited the community dataset and found 25 duplicate contracts and 26 that did not resemble Ponzi schemes. These findings reduced the dataset to 133 Ponzi contracts, composing the SADPonzi benchmark, which was made publicly available (Chen et al., 2021).

Other authors chose not to use this dataset as a basis, instead collecting their own data. One such case is Yu et al. (2021), who sourced 50 Ponzi contracts from community-reported tags on Etherscan, then extended this labelled dataset to 250 using XBlock (Yu et al., 2021). Zheng et al. (2023) also collected their own ground truth dataset. This process was done by crawling Etherscan for open-source contracts and manually inspecting the source code for Ponzi logic. After collecting it, the authors reported having 6498 contracts, of which 314 were Ponzi Schemes. This dataset was also used by Liang et al. (2025) (Liang et al., 2025; Zheng et al., 2023).

This study draws from 2 of the described sources: the SADPonzi ground truth set, which provides 133 formally verified Ponzi contracts, and Zheng et al.'s set, which provides 314 Ponzi contracts.

The features used by the authors had two possible motivations: either they were code-based features that enabled contract classification without transactions, or transaction-based features that helped classify contracts with hidden code.

Chen et al. (2018) drew from the operation codes of the contract bytecode. The code features consisted of the frequency of opcodes found in their dataset, on the assumption that Ponzi Contracts would need to implement certain characteristic patterns to replicate the distribution logic of the fraud (Chen et al., 2018). Jung et al. (2019) would extend on the opcode approach by building a "zero-day model", a sort of classifier that could operate before any transactions occurred. The code features of this model included SSTORE, POP, MSTORE, SWAP1 and JUMP (which are instructions native to the Ethereum Virtual Machine that all contracts use to execute their logic).

Beyond using opcode frequencies, other authors have used bytecode similarity. Zhang et al. (2022) employed the Normalised Levenshtein distance to compare contracts, assuming that the redistribution logic of Ponzi Scheme contracts is conceptually narrow. Consequently, Ponzi contracts should have similar bytecodes. When evaluating feature importance in their model, bytecode similarity ranked highest among all features (Zhang et al., 2022).

Zheng et al. (2023) extract features that do not rely on transaction records. They analyse bytecode to compute opcode frequencies and TF-IDF (Term Frequency-Inverse Document Frequency) scores, then extend these to N-gram sequences to identify local sequential patterns. They also generate Word2Vec embeddings of the opcode sequence to capture semantic relationships among instructions. A creator vector, derived from the contract

creation graph, indicates whether the deploying address has previously deployed a verified Ponzi contract, making this information available at deployment time, before any user has interacted with the contract (Zheng et al., 2023).

Finally, SADPonzi uses symbolic execution to extract semantic information from the contract bytecode. To accomplish this, the algorithm detects investing and rewarding actions. By analysing the structure of these actions, the model classifies contracts based on the formal criteria for a Ponzi scheme.

Transaction-based works base their feature sets on the patterns produced by a Ponzi scheme's transactions. Chen et al. (2018) used 7 features to capture these patterns: the number of investments and payments, the contract balance, the known rate (proportion of payment recipients who had previously invested), the paid rate (proportion of investors who received at least one payment), the difference index measuring the skewness of the payment-minus-investment distribution across participants, and the maximum payment count to any single participant. These features can capture the core mechanics of a Ponzi scheme, in which early investors receive disproportionate returns funded entirely by the investments of later investors (Chen et al., 2018). Subsequent work would extend this feature set to 13 features, including means, standard deviations, and various skewness measures between payment counts and investment counts across a contract's participants. Despite adding these new features, the study found that using only transaction features underperformed compared to the code-based features, which the authors attributed to many contracts having few or no transactions (Chen et al., 2019).

Jung et al. (2019) introduced new features, including the Gini coefficients for inflow and outflow amounts, transaction timing, the average time between transactions, the overlap between addresses that both paid into and received from the contract, and the number of unique addresses on each side. This study revealed that Ponzi contracts generally had a significantly longer average interval between transactions than Non-Ponzi Contracts (Jung et al., 2019). This finding suggests that temporal-based features could serve as a possible discriminator between the two labels.

Hu et al. (2021) developed 14 time-series features from transaction records without requiring source code access. These features include contract balance, Ether inflows and outflows (distinguished between external and internal transactions), transaction counts, and unique address counts for each transaction type. Each feature is aggregated over a fixed time window, creating a time-series view of each contract's activity.

Another approach to the transaction record was the one used by Yu et al. (2021). This study constructed a transaction network in which contracts and external accounts are nodes and where individual transactions are directed edges. Node features were computed from the transaction record and fed into a graph convolutional network (GCN) that could aggregate both the node features and the neighbourhood structure (Yu et al., 2021).

Liang et al. (2024;2025) contend that features based on code and transactions show little correlation with Ponzi scheme behaviour. Instead of merely extracting features, they built a Contract Runtime Behaviour Graph (CRBG) by invoking contracts within an instrumented EVM and tracking execution dynamics through dynamic taint analysis. This process produces a graph that illustrates control-flow edges (connecting sequentially executed opcodes) and data-flow edges (linking taint sources and sinks). Node features are generated using Doc2Vec embeddings of each opcode description, as outlined in the Ethereum yellow paper. Essentially, this method analyses a contract's behaviour itself rather than relying on a summary of its code or transaction history (Liang et al., 2024, 2025).

This study focuses solely on transaction-based features, it does not include opcodes or bytecodes, ensuring the model remains effective without them. The temporal dimension of prior work is extended by measuring the rate of decay in investor recruitment. This metric aims to capture the central collapse mechanism of a Ponzi scheme, in which the absence of new investors prevents payment to prior investors.

2.2 Model Performance and Research Gap

Regarding the models and their results, Chen et al. (2018) employed an XGBoost model with three configurations: one using only transaction features, another with only code features, and a third combining both types. Research concluded that the best-performing model was the mixed-feature configuration, achieving a precision of 0.96 and an F1-score of 0.86 (Chen et al., 2018). Chen et al. (2019) would alter the model to a Random Forest, finding that it reduced variance. The best model achieved a precision of 0.95 and an F1-score of 0.79 using both transaction and code features (Chen et al., 2019).

Jung et al. (2019) developed benchmarks involving three models, each with three configurations. The benchmark included a J48 model, Random Forest, and Stochastic Gradient Descent, tested with code-only features, behaviour-based features, and a combination of both. The full-feature J48 model achieved a precision of 0.98 and an F1-score of 0.97 (Jung et al., 2019).

The LSTM trained on 14 features, as deployed by Hu et al. (2021), produced results separated by contract type. Its F1-score ranged between 0.70 and 0.83, and its precision ranged between 0.795 and 0.925. The authors acknowledged that the LSTM had a relatively small training dataset, which led them to slice individual contracts to generate more data points. This, in turn, meant that individual slices no longer represented a contract's life cycle (Hu et al., 2021).

Zhang et al. (2022) proposed an improved version of the LightGBM algorithm combined with SMOTE-TOMEK mixed sampling to handle class imbalance. The model used transaction features, opcode frequency, and bytecode similarity, and achieved a precision and F1-score of 0.97 (Zhang et al., 2022).

Zheng et al. (2023) preferred to use a Multi-View Cascade Ensemble (MulCas) a type of model that trains separate classifiers on each feature type and combines them through a cascade and voting mechanism. The base learners used were XGBoost and Ridge. The best result obtained by these authors was a precision of 0.95 and an F1-score of 0.79 (Zheng et al., 2023).

SADPonzi achieved 100% F1-Score, recall, and precision, highlighting its distinct approach compared to traditional machine learning classifiers. However, this benchmark has a limitation: it can only identify four scheme types it has been trained on, based on bytecode. The study also found that when the code was obfuscated, the ML baselines performance declined, whereas SADPonzi remained stable (Chen et al., 2021).

Yu et al. (2021) compared the GCN+Feature model with feature-only baselines such as Logistic Regression, SVM, and Random Forest. The study showed that the GCN model achieved F1-scores of 0.89-0.90 across all three datasets as well as recall values of 0.92-0.94, consistently outperforming the feature-only random forest baseline, which had precision values between 0.80-0.87 and an F1-score of 0.77-0.81 (Yu et al., 2021).

Lastly, there is the model applied by Liang et al. (2024;2025). These authors used a Graph Attention Network (GAT) applied to the CRGB. The 2025 results report a precision of 0.97 and an F1-score of 0.98. Although these are remarkable results, they are achieved on contracts with available source code, which is a small and unrepresentative fraction of all deployed contracts (Liang et al., 2024, 2025).

The literature consistently favours tree-based ensemble methods, with Random Forest and XGBoost emerging as the dominant classifiers in most studies. In every study that tests both code-based and transaction-based features, combined feature sets outperform transaction-only configurations, a finding reported by Chen et al. (2018), Chen et al. (2019), and Jung et al. (2019). A key limitation of code-based methods is their reliance on access to bytecode or source code. Techniques like opcode frequency analysis, bytecode similarity, and SADPonzi's symbolic execution all depend on having the contract's compiled code, which restricts their use to a small subset of contracts with verified source code. Transaction-based approaches avoid this constraint, but existing work captures only aggregate statistics over a contract's lifetime, missing the temporal dynamics of investor recruitment. Jung et al. (2019) noted that Ponzi contracts exhibit longer average intervals between transactions, suggesting a temporal signal exists, but no study explicitly models the rate at which new investor recruitment accelerates and subsequently decays. This rate is the central mechanism by which Ponzi schemes collapse. This study addresses that gap by constructing transaction graph features that capture both the structural and temporal dimensions of investor recruitment without requiring access to contract code.

3 METHODOLOGY

This chapter documents the methodologies applied to develop the project. The approach undertaken relies exclusively on transaction data, there is no source code, bytecode analysis or manually crafted sets of rules for Smart Ponzi Scheme detection. Previous works, such as Chen et al. (2018), relied on bytecode disassembly or Solidity source code, which is available for only a minority of contracts. This transaction-only approach is motivated by these studies and the difficulties encountered by the researchers.

The central hypothesis is that Smart Ponzi schemes exhibit structural patterns that distinguish them from normal DApps, and that these patterns can be used to create network features for a machine-learning classifier.

The pipeline proceeds in three stages. Firstly, the data is loaded from two separate CSV files into a unified, chronological, direction-aware dataframe. Secondly, the data is checked for inconsistencies, and contract-level features are engineered. Third, four supervised classifiers are trained with cross-validation and hyperparameter search, then evaluated on a held-out test set.

The classification type and models were chosen based on the literature. Rule-based approaches, such as Bartoletti et al.'s bytecode similarity, were not considered because they couldn't cover all possible cases, as well as required domain-specific knowledge (Liang et al., 2024). By contrast, supervised learning can learn the discriminating boundary from labelled examples and generalise to novel scheme designs that follow the same structural patterns.

3.1 DATA

The data comprises on-chain Ethereum transaction records for two labels (Ponzi or DApp). The ground truth labels were drawn from two different sources. The initial data was obtained from the SADPonzi paper by Chen et al. (2021), which provides manually verified Smart Ponzi contracts. This dataset was then expanded using data from Zheng et al. (2023), which captures new and larger-scale Smart Ponzi contracts. The final combined dataset consists of 311 labelled Ponzi contracts and 1228 normal DApp contracts.

The transaction data was retrieved from Google BigQuery's public Ethereum dataset. The data was queried for the period from 1st January 2015 to 1st January 2023. Inflow transactions were recorded in the "crypto_ethereum.transactions" table, which stores externally initiated contract calls. Outflows were recorded in the "crypto_ethereum.traces" table, which records internal calls emitted by contracts during execution (*Ethereum in BigQuery*, n.d.). Transactions were capped at 1000 per direction. This threshold was selected based on the typical lifecycle of Ponzi contracts, which tend to be short-lived. The median Ponzi contract in this dataset has only 27 total transactions, and the vast majority are exhausted well below the cap. This was achieved using a window function in the query. This function partitioned the data by contract

and ordered it chronologically. This process yielded four CSV files that had to be concatenated to create the full dataset.

Each transaction recorded the contract address, counterparty address, transferred value in ETH (converted from Wei at query time via division by 10^{18}), block timestamp, and transaction hash. Before analysis, all files were transformed to follow a unified schema, enabling their concatenation into a single dataset.

In this combined dataset, timestamps were converted to datetime objects, and any rows that could not be converted were removed, as all subsequent analysis relies on a chronologically ordered sequence of transactions. Within each contract, transactions were sorted by date and assigned a sequential rank, and this process was applied separately to inflows and outflows to capture direction-specific temporal features. This resulted in a transaction dataset that served as the foundation for the exploratory analysis and feature engineering, with data aggregated into a single row per contract to create the feature matrix for classification. This finalised dataset produced the following statistics:

Table 1 - Data Dimensions and Values

Dimension	Value
Total records	1289157
Unique Contracts	1539
Ponzi Contracts	311
Normal DApps	1228
Inflow records	923945
Outflow Records	365212
Median transactions per Ponzi	27
Ponzi Contracts with no Outflows	42 (13.5%)
DApp Contracts with no Outflows	564(45.9%)

A notable metric is the count of DApps with no outflows. This reflects the source ("crypto_ethereum.traces"), as the table only tracks ETH transferred via internal contract calls and does not capture ERC-20 token transfers. As a result, DApp contracts that distribute value exclusively through token transfers will appear to have no outflows in this dataset. Some valid DApp contracts receive ETH from users but do not distribute it programmatically, resulting in no outflows.

3.2 Transaction Graph Construction

The dataset reflects a directed, weighted, temporal graph $G = (V, E)$. Where V is the set of nodes and E is the set of edges. The nodes comprise the contract addresses and all addresses of external accounts that interacted with them. Edges represent the individual transactions. An inflow is an edge directed from an external account to a contract, an outflow is an edge

directed from the contract to the external account. Each edge has two attributes: its ETH value and the block timestamp, which makes the graph weighted and temporal.

Each contract constructs a subgraph, meaning the analysis is performed at the level of a local transaction neighbourhood. This design choice reflects the classification objective, since the task is to assign a label to each contract.

This graph representation is motivated by the expected structure of a Ponzi scheme. Ponzi schemes are expected to accumulate inflows while maintaining relatively low outflows, as many participants invest while only a few are paid out under the contract. Additionally, the recruitment rate of new investors is typically high early in a contract's lifecycle and then decreases as the recruitment pool is depleted. This behavioral pattern helps differentiate Ponzis from standard DApps.

3.3 Feature Engineering

In the feature engineering section, the initial dataset of approximately 1.3 million rows was aggregated by contract to create contract-level features. The features were created through a three-pillar process based on the literature from the second chapter, exploratory data analysis, and some trial and error, ultimately leading to the final feature set. The features presented in the table below are those that participated in the feature selection process.

Table 2 - Results from Feature Engineering

Feature	Definition
in_degree	Number of unique addresses in inflow transactions
out_degree	Number of unique addresses in outflow transactions
in_out_ratio	Ratio of unique inflow addresses to unique outflow addresses, with one added to the denominator to avoid division by zero
in_degree_per_tx	Proportion of a contract's total transactions that originate from distinct addresses
repeat_sender_ratio	Proportion of unique inflow addresses that sent more than one transaction to the contract
max_sender_ratio	Proportion of total inflows sent by the single most active sender
new_sender_rate	Proportion of inflow transactions in which the sender appeared for the first time, computed in chronological order.
new_sender_decay	Difference in the log-transformed rate of new senders between the second and first half of a contract's transaction history. Negative values indicate that recruitment slowed over the contract's lifetime
has_outflows	Binary indicator, 1 if a contract has at least one unique outflow address, 0 otherwise
total_txs	Total transactions (used for normalisation)

Some features need additional motivation beyond the description provided in table. *in_degree_per_tx* was added because Ponzi schemes, which often attract many different investors each sending a single payment, are expected to show a higher ratio of unique senders to total transactions than typical DApps, where the same address might interact repeatedly, resulting in a higher *in_degree_per_tx* for Ponzi contracts relative to DApps. *max_sender_ratio* was introduced to test if a dominant sender is responsible for a disproportionate share of inflows. *new_sender_decay* aims to detect recruitment collapse as the scheme grows. The logarithm is applied to compress the scale of the rates and ensure that large absolute differences in high-activity contracts do not dominate over the same proportional decay in smaller ones. A negative value indicates that the rate of new senders in the latter half of transactions is lower than in the first half, a behaviour expected in Ponzi schemes. Conversely, zero or positive values indicate steady participant arrival, which aligns

more with legitimate DApps. Lastly, *has_outflows* is a binary flag motivated by the observation that most Ponzi contracts are more likely to have outflows than DApps, making the presence of outflows a potential discriminator.

3.4 Data Cleaning and Splitting

Before feature selection, the dataset was cleaned in three steps. First, any infinite values caused by edge cases in feature engineering were replaced with NaN. Second, data quality checks verified contract counts, class distribution, and the absence of missing values. Lastly, a filter is applied to the data to remove contracts with fewer than five unique senders, as they cannot yield meaningful graph-based features. *new_sender_rate* and *new_sender_decay* are essentially undefined at this scale, while *repeat_sender_ratio* is essentially zero. This filter leaves the dataset with 1190 contracts, of which 167 are Ponzi schemes, and 1023 are normal Dapps.

After completing these checks, the data was split into training and testing sets using an 80/20 split with stratified sampling to maintain class imbalance, resulting in 952 training contracts (134 Ponzi, 818 DApps) and 238 test contracts (33 Ponzi, 205 DApp).

Finally, missing values were imputed with the median of each column to preserve the distribution, and the test set used the training set medians to prevent data leakage.

3.5 Feature Selection

After splitting the data, the next step was feature selection to identify the most informative features for subsequent modelling. This process was carried out using a cross-validated mutual information approach. For each of the five stratified folds, features were ranked according to their Mutual Information score, and the five highest-ranked features were recorded. Features were selected using a "voting" method, meaning that only features selected in at least 80% of the folds (4 out of 5) were retained for subsequent modelling.

Mutual information was chosen as the selection criterion because it is model-agnostic. The objective was to avoid favouring any specific classifier during feature selection, ensuring a fair comparison between models. Performing this selection independently within each fold also prevents the validation set from influencing feature ranking, reducing the risk of information leakage during model evaluation.

Table 3 - Results from Feature Selection

Feature	"Votes"	Selected
in_out_ratio	5	Yes
repeat_sender_ratio	3	No
new_sender_rate	0	No
new_sender_decay	5	Yes
in_degree	4	Yes
in_degree_per_tx	0	No
out_degree	5	Yes
max_sender_ratio	0	No
has_outflows	3	No

Votes indicate the number of cross-validation folds (out of 5) in which a feature ranked among the top five by mutual information score. Features with four or more votes were selected for modelling.

The results in Table 3 the chosen features: *in_out_ratio, new_sender_decay, out_degree and in_degree*.

3.6 CLASSIFICATION MODELS

Four classification models were trained: logistic regression, random forest, XGBoost, and a support vector machine. These models were selected because they had been previously tested by other approaches discussed in the literature. Both Chen et al. (2018) and Zheng et al. (2023) used XGBoost, Chen et al. (2019) used random forest and logistic regression serves as a baseline model to verify whether the feature space is linearly separable.

To ensure a fair comparison, all models received the same features and were trained using the same stratified 80/20 split. In all models, hyperparameter optimisation was performed using RandomizedSearchCV with five-fold cross-validation. The optimisation metric chosen was ROC-AUC due to its insensitivity to class imbalance. Cross-validation was used exclusively on the training set.

Class imbalance was handled differently across model types. Logistic regression, random forest, and SVM applied the *class_weight='balanced'* parameter, which adjusts loss contributions inversely to class frequency. In contrast, XGBoost used *scale_pos_weight* set to the ratio of negative to positive training samples, providing a similar reweighting strategy within its gradient boosting method.

Logistic regression was trained with StandardScaler, which is crucial because features operate on different scales. The search space covered five values of the regularisation parameter C (0.01 to 100), two solvers (lbfgs, liblinear), and an L2 penalty, resulting in ten combinations in



total and constituting a full grid search. The selected configuration used C=100 with the lbfgs solver.

Random forest is scale-invariant and requires no preprocessing. Fifty random configurations were sampled from a search space covering the number of estimators (100–500), maximum tree depth (5–20 or unconstrained), minimum samples for splitting and leaf nodes, feature sampling strategy, and bootstrap sampling. The selected configuration used 300 estimators with a maximum depth of five, a minimum number of samples to split of five, a minimum sample size of two, sqrt feature sampling, and bootstrapping.

XGBoost was configured with scale_pos_weight set to the training imbalance ratio. Thirty configurations were sampled from a space covering tree depth (3–7), learning rate (0.01–0.1), subsampling ratios (0.6-1), *colsample_bytree* (0.6-1), minimum child weight (1-10), number of estimators (100-300) and gamma regularisation(0-0.3). The selected configuration used 300 estimators with a maximum depth of 3. The learning rate was set at 0.01, the subsample value at 0.8 column subsampling ratio of 0.6, with a minimum child weight of 5 and a gamma regularization of 0.1.

While Random Forest and XGBoost partition the feature space through decision trees, SVM finds a maximum-margin boundary between classes. Since the SVM is sensitive to feature scales, StandardScaler was applied, and probability estimates were enabled to compute ROC-AUC. Twenty random hyperparameter configurations were sampled and tested across the 5 folds covering C (0.01-100), kernel type (RBF or Linear), and gamma. The selected configuration used a linear kernel and a C value of 100, gamma is not used by the linear kernel and is therefore disregarded.

To address the class imbalance, SMOTE was evaluated as an alternative to model weights. Rather than re-weighting misclassification costs, SMOTE generates synthetic observations for the minor class. This technique was applied to the full training set, resampling from 952 contracts (134 Ponzi and 818 Dapp) to a balanced set of 1636 contracts (818 of each class). To ensure that the models with SMOTE did not address the issue of class imbalance twice (once through SMOTE, another through class weights), class weighting was disabled in the SMOTE variants, with all other hyperparameters kept identical to ensure a fair comparison.

4 RESULTS AND DISCUSSION

4.1 EXPLORATORY DATA ANALYSIS

Before aggregating the data to build the network features used for classification, an exploratory analysis of the transaction data was conducted. This section presents the key findings of this analysis, highlighting aspects that influenced the features used in the classification models.

The original ground truth dataset contained 311 Smart Ponzi Contracts and 1228 normal DApp contracts, a ratio of approximately 1 to 4. At the transaction level, this imbalance becomes even more severe, since legitimate DApps have more activity than Smart Ponzi Schemes. This imbalance persists even with the 2000 transactions-per-contract cap, but because classification is performed only at the contract level, only the 1:4 ratio is relevant, and it is addressed at the modelling stage.

Transaction values are heavily skewed for both classes. Normal DApps have a median of 0 (because 45.9% of contracts have no outflows) and a mean of 7.84 ETH. The maximum transaction is 820000 ETH, referring to Golem Network's initial offering of 82% of their supply (*What Is Golem?*, n.d.). The median value once again shows that ETH value alone is not a good discriminator, as previously mentioned, because legitimate DApps might not have outflows and therefore "artificially" skew the data. Regarding Smart Ponzi outflows, the median amount is 0.021 ETH, the mean is 1.93 ETH, and the maximum reaches 1098 ETH.

The Gini Coefficient was calculated for each contract to assess the inequality in ETH value distribution among its counterparties. This coefficient was calculated by ranking counterparties by total ETH received and measuring how far the actual distribution deviates from perfect equality, where zero indicates all counterparties received the same amount and one indicates a single counterparty received everything. Chen et al. previously observed that Smart Ponzi schemes often have low contract balances because they pay out investors rapidly (Chen et al., 2018). If this concentration influences the ETH value distribution, it should be detectable through inequality measures. To test this hypothesis, the contracts were divided into three separate bins based on the quantiles of transaction count.

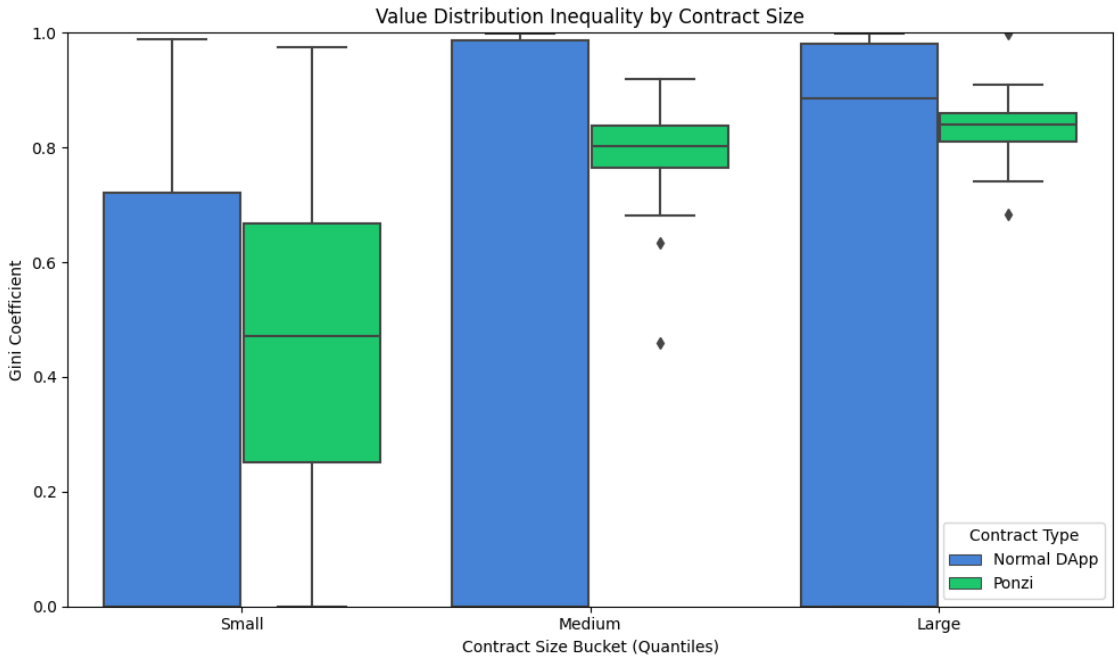


Figure 1 - Value Distribution Inequality by Contract Size

The results indicate that for small contracts, both classes reach the extremities of the plot without clear separation. In medium and larger-scale contracts, Smart Ponzi Schemes tend to have higher Gini coefficient values, aligning with the expected concentration of payments to earlier investors in the scam. DApp contracts, however, show significant variability within the same bins, covering the full range in both cases. In the large bin, the median Gini value for DApp contracts is higher than in the medium bin, suggesting that larger DApps tend toward greater value concentration.

These results imply that the Gini coefficient is not an effective discriminator. Legitimate DApps can also exhibit unequal fund distributions, which can produce high Gini values without fraudulent intent. Consequently, the Gini coefficient was ultimately excluded from the final feature set. This also aligns with previous findings that emphasise the importance of structural patterns over the amount of ETH transferred between addresses, further reinforcing the decision to focus on graph structural features.

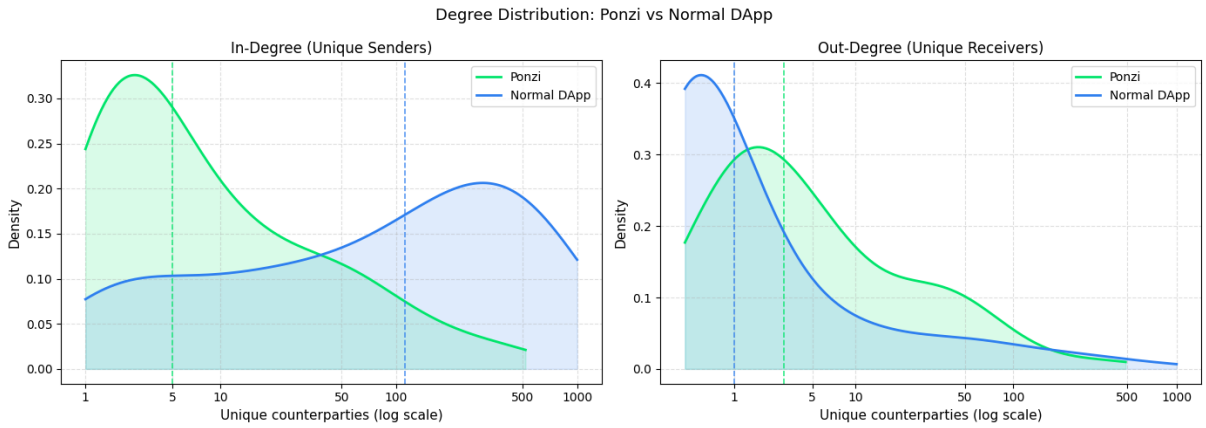


Figure 2 - Degree Distribution: Ponzi VS Normal DApp

Figure 2 illustrates the in-degree and out-degree distributions for both classes on a log scale, with the median values marked by dotted lines. Ponzi contracts tend to have a concentrated in-degree distribution, peaking at around 5 unique senders, indicating short-lived activity and a small participant base. In contrast, normal DApps show a broader distribution, primarily between 100 and 500 unique senders, which aligns with their tendency to attract users over longer periods. Regarding out-degree, both classes peak at low values (under 5). Ponzi contracts show a slightly higher median out-degree compared to DApps. This difference partly results from a data-collection limitation: outflows from DApps via ERC-20 token transfers are not captured in the *crypto_ethereum.traces* table, which records only ETH-based internal calls. As a result, the out-degree for many DApps might be underestimated.

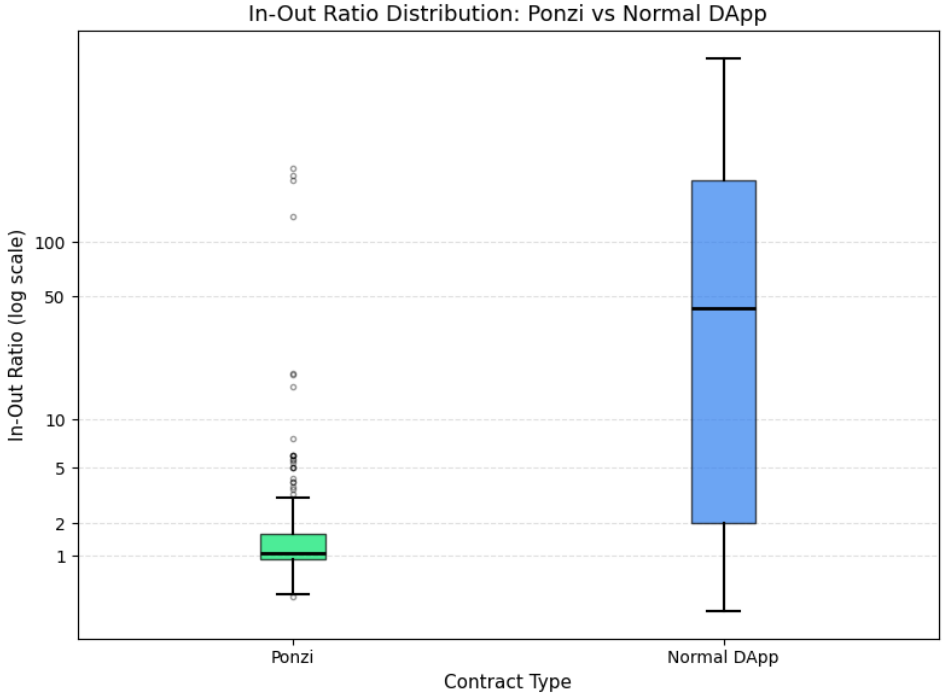


Figure 3 - In-Out Ratio Distribution: Ponzi VS Normal DApp

Figure 3 shows the in-out ratio distribution for both classes. Ponzi contracts cluster tightly around a ratio of approximately one, with very low variance. Normal DApps show a median of around 45 and a large spread across the full range. Ponzi schemes might be expected to have higher in-out ratios, given that their functioning requires them to receive more investments than they pay out to investors. The explanation once again lies in the same ERC-20 limitation noted above. Because DApp token-based outflows are invisible in the traces table, their measured out-degree is underestimated, resulting in inflated in-out ratios. Despite this, the in-out ratio remains the strongest feature for the classifier. This is due to the tight clustering of Ponzi contracts around one, which is a distinguishing pattern. Ponzi schemes tend to have roughly equal numbers of unique senders and receivers, reflecting their redistribution mechanism, whereas DApps exhibit far greater structural variability.

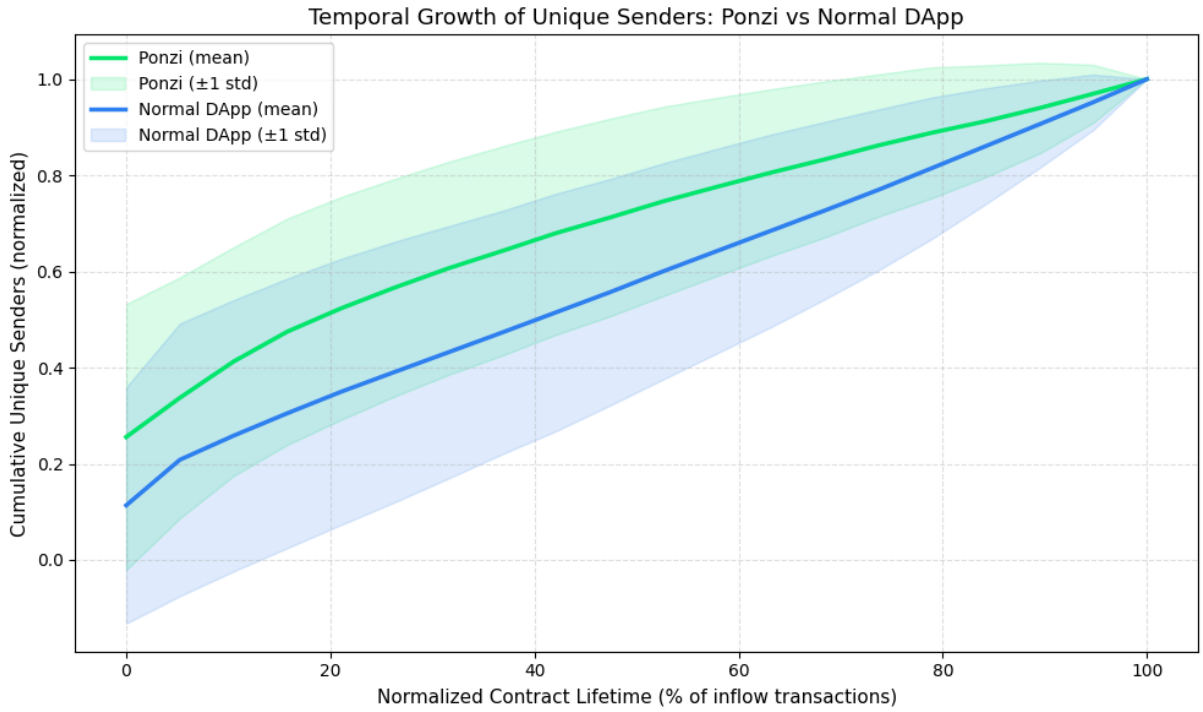


Figure 4 - Temporal growth of Unique senders: Ponzi VS Normal DApp

Figure 4 presents the cumulative proportion of unique senders recruited over each contract's normalised lifetime. Ponzi contracts exhaust their unique sender pool faster. When a Ponzi reaches the midpoint of its transaction history, it has already recruited approximately 70% of its total unique senders. Normal DApps have a slower and more linear growth curve. This reflects a legitimate investor acquisition that grows naturally over time. This difference in recruitment patterns motivates adding *new_sender_decay* as a feature, which represents the difference in the new-sender rate between the first and second halves of a contract's inflow history.

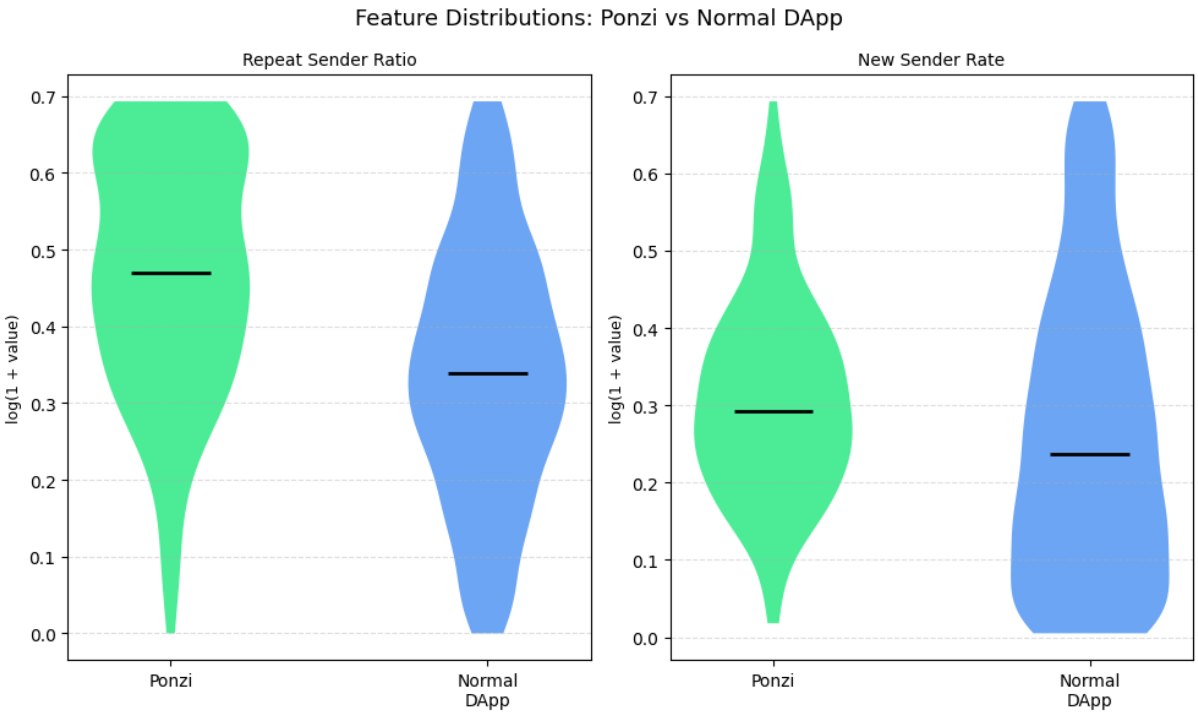


Figure 5 - Feature Distributions: Ponzi VS Normal Dapp

Figure 5 illustrates the distributions of the repeat-sender ratio and the new sender rate for both classes. The Ponzi class exhibits a higher repeat-sender ratio, indicating a larger proportion of inflow counterparties that engage in multiple transactions. This possibly reflects reinvestment strategies or fund recycling by the operator. Although the median new-sender rate is marginally higher for Ponzi contracts, the substantial overlap between their distributions diminishes its usefulness as an isolated indicator.

4.2 Classification Models

This section presents the model results based on a filtered set of 1190 contracts, including 167 Ponzi contracts and 1023 normal Dapps. The dataset was divided using an 80/20 stratified split, maintaining the Ponzi/Dapp ratio across the partitions. As a result, the test set contains 238 contracts, of which 33 are classified as Ponzi contracts. Four metrics are reported for each model: AUC, Precision, Recall, and Macro F1. AUC measures the discrimination ability regardless of the classification threshold. Macro F1 is preferred over accuracy because it weights each class equally, preventing the majority Normal DApp class from hiding poor performance on the minority Ponzi class. Precision and Recall are included to detail where each model performs well or poorly, specifically within the Ponzi category.

Table 4 - Classification Results for Ponzi Label

Model	AUC	Precision	Recall	Macro F1
Logistic Regression	0.902	0.366	0.909	0.685
Random Forest	0.925	0.583	0.848	0.814
XGBoost	0.926	0.566	0.909	0.816
SVM	0.918	0.317	0.970	0.636

All models reported high recall values (0.848–0.970), while precision remained low (0.317–0.583) across the board. The results reflect a tendency to flag most Ponzi contracts at the cost of also misclassifying a large share of Normal Dapps. Logistic Regression performed at the level of the SVM, though these were the worst performing models. The SVM achieved the highest recall of all models at 0.970, meaning it correctly identified 97% of Ponzi contracts in the test set, but also reported the lowest precision at 0.317, meaning that fewer than one in three contracts flagged as Ponzi were actual Ponzi schemes. In practice, such a precision-recall trade-off results in a high rate of false positives. Random Forest and XGBoost achieved the highest Macro F1 scores at 0.814 and 0.816, respectively, demonstrating the most balanced performance across both classes and making them the two best-performing models overall, consistent with their use in the Ponzi detection literature.

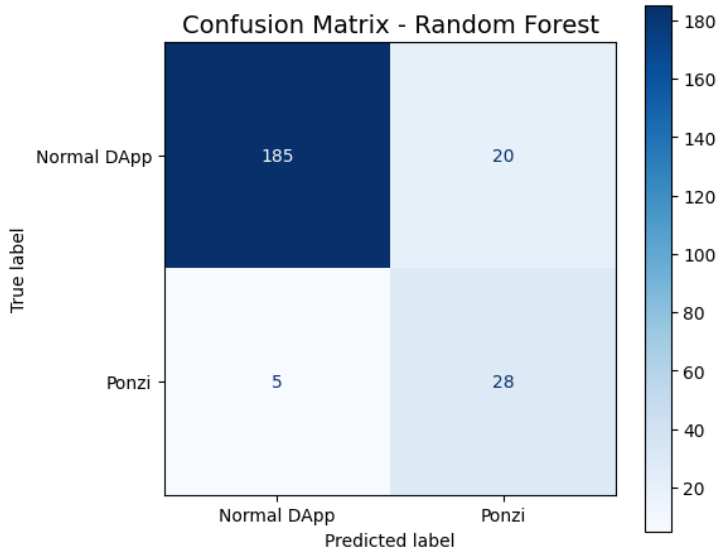


Figure 6 - Confusion Matrix: Random Forest

The Random Forest model correctly classified 213 of 238 test contracts, identifying 185 Normal DApps and 28 of 33 Ponzi contracts correctly, as shown in Figure 6. There are five

contracts that were wrongly classified as Dapps (false negatives), while 20 Normal DApps were incorrectly flagged as Ponzi (false positives). The model achieves an AUC of 0.925 and a Macro F1 of 0.814, indicating strong overall discrimination and a moderate false-positive rate.

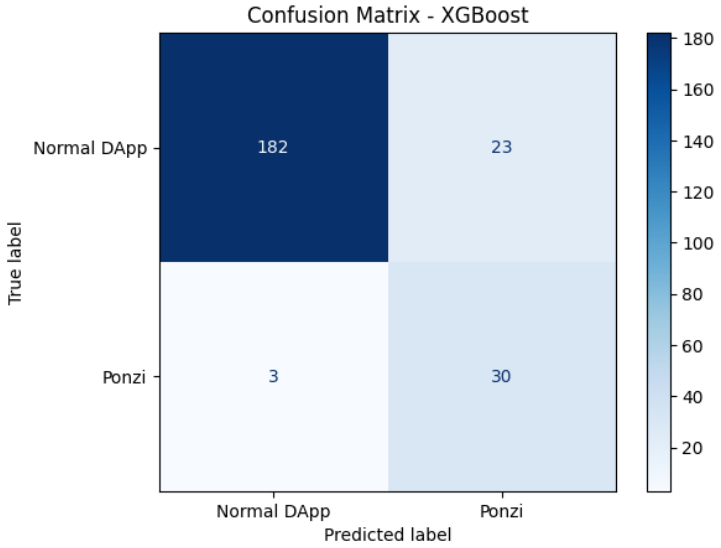


Figure 7 - Confusion Matrix: XGBoost

XGBoost accurately classified 212 out of 238 contracts, correctly identifying 30 of 33 Ponzi schemes and 182 Normal DApps. It missed only 3 Ponzi contracts but incorrectly labelled 23 DApps as Ponzi, slightly outperforming the random forest. The model achieved an AUC of 0.926 and a Macro F1 score of 0.816, surpassing the Random Forest on both metrics. The two models show a trade-off: XGBoost detects one additional Ponzi contract but produces three more false positives.

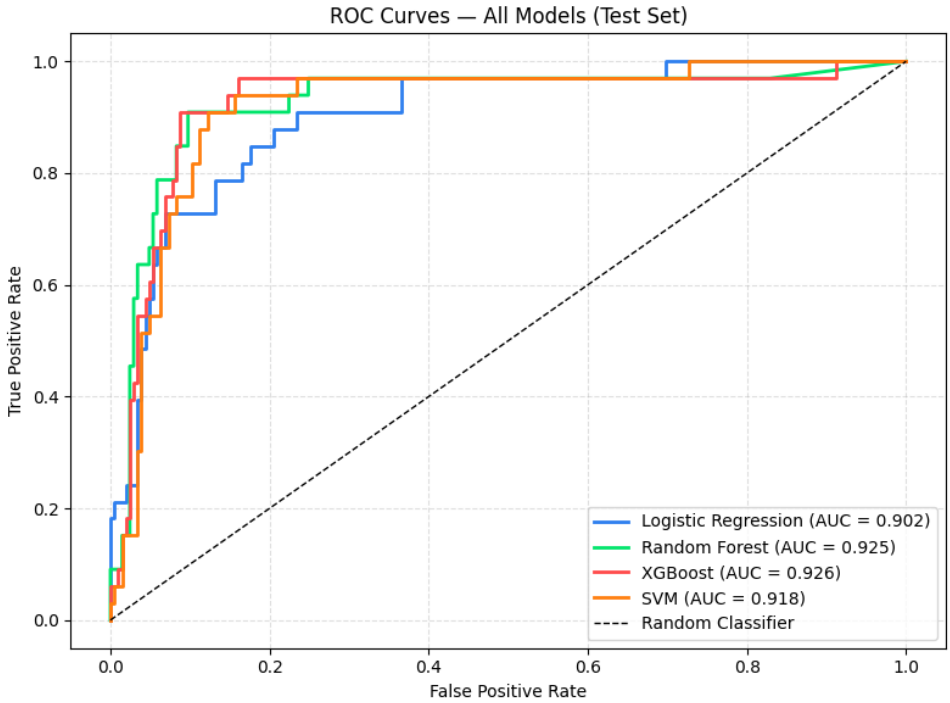


Figure 8 - ROC curve comparison

Figure 8 presents the ROC curves for all four models on the test set. Each model demonstrates strong discrimination ability, with AUC values from 0.902 for Logistic Regression to 0.926 for the XGBoost model. The curves are closely clustered, except for the Logistic Regression, which has a notably lower curve.

Looking at the misclassifications of the best performing models, both the Random Forest and XGBoost models misclassified 22 contracts: 19 normal DApps and 3 Ponzi schemes. The 19 misclassified DApps have feature values similar to Ponzi contracts, with *in_out_ratio* near one and negative *new_sender_decay* values. These values indicate they are legitimate contracts with transaction graph structures that resemble Ponzi patterns. Regarding the three missed Ponzi contracts, one had no outflows, making it resemble the many DApps that also record none. The second scheme had a low *in_out_ratio* of 0.244 and more outgoing addresses than incoming ones, indicating an unusual payout structure not typical of Ponzi schemes. The last scheme had 12 unique senders and one receiver which may have contributed to its misclassification.

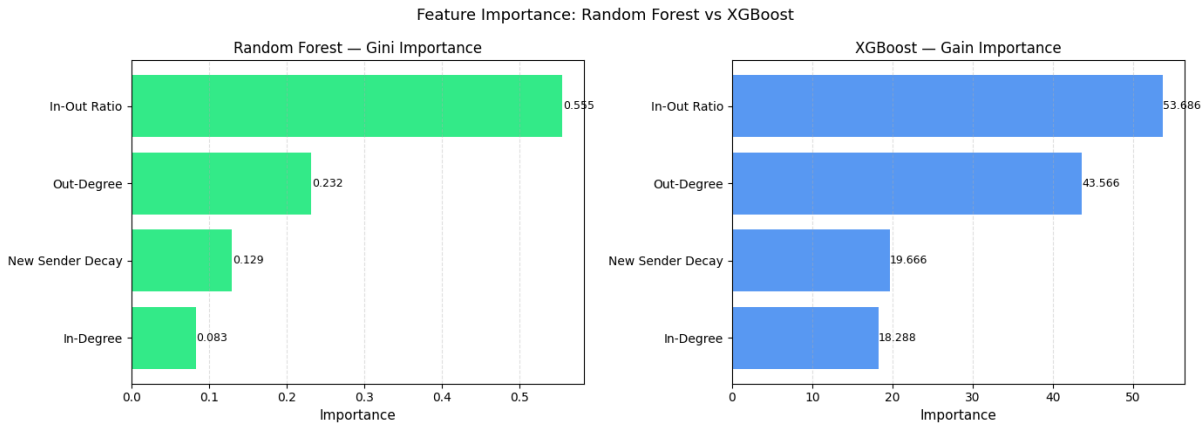


Figure 9 - Feature Importance: Random Forest vs XGBoost

In Figure 9, the feature importances for the Random Forest and XGBoost models are shown. The two metrics aren't directly comparable, as one is calculated using Gini Importance (Random Forest) and the other using Gain Importance (XGBoost). Gini Importance measures the average reduction in node impurity across all trees of the random forest model, whilst Gain Importance measures the average improvement in the loss function a feature brings across all splits. Despite this difference, both models agree on the ranking of all features with the most important features being In-Out Ratio (by a large margin). The second most important feature is Out-Degree, followed by New Sender Decay and In Degree. The dominance of In Out Ratio can possibly be explained by the ERC-20 limitation that was previously discussed. New sender decay is the only temporal feature used by the model confirming that the dynamics of investor recruitment carry discriminatory power.

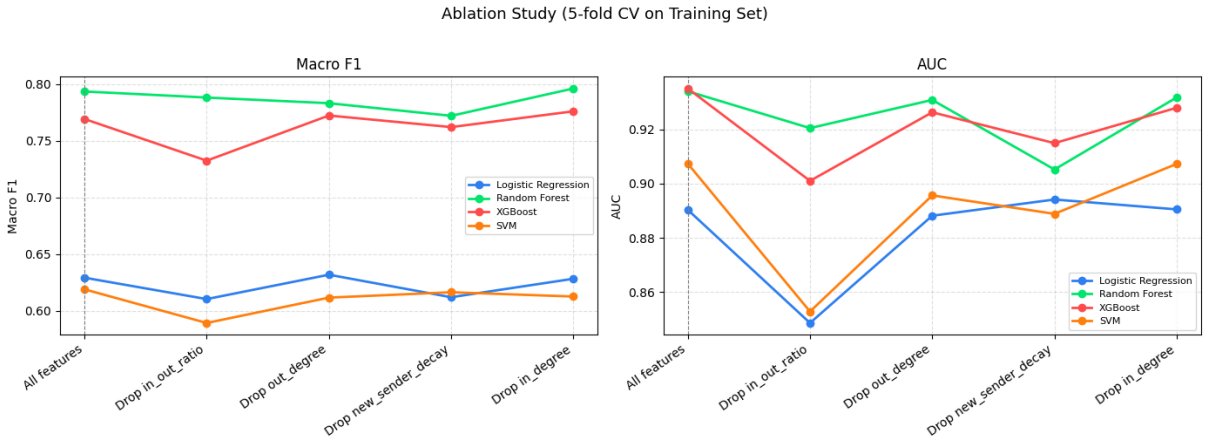


Figure 10 - Ablation Study

Figure 10 presents the ablation study, showing the results of removing each feature individually on Macro F1 and AUC across all four models, using five-fold cross-validation on the training set. *in_out_ratio* is the feature that produces the largest impact when dropped, especially when examining the AUC values for Logistic Regression and SVM. For the Random Forest model, the AUC drops substantially when *new_sender_decay* is dropped. This drop also occurs for XGBoost, albeit to a less significant extent. The plot shows that, more than being

the best-performing models, Random Forest and XGBoost are also the more stable ones, suffering less pronounced drops compared with the other two models. Still, each model suffers at least one large drop; for XGBoost, this drop is connected to *in_out_ratio* and is present in both Macro F1 and AUC values. For Random Forest, this drop is present when looking at *new_sender_decay*; although it is present in the Macro F1 value, it is largely more impactful in the AUC value, with it being the largest drop of the two best-performing models.

Table 5 - Model comparison with other relevant studies

Study	Method	Features	Precision	Recall	F1 (Ponzi class)
Chen et al. (2018)	XGBoost	Transaction + Bytecode	0.94	0.81	0.86
Hu et al. (2021)	LSTM	Transaction only	0.80-0.93	0.55-0.78	0.69-0.83
YU et al. (2021)	GCN + Features	Transaction + Graph	0.87	0.92-0.94	0.89-0.90
This work	Random Forest	Transaction only	0.58	0.85	0.69
This work	XGBoost	Transaction only	0.57	0.91	0.70

Table 5 presents the results of the current model alongside those from relevant literature. Model complexity and feature sets must be taken into account to ensure a fair comparison. The current work uses two models previously reported in the literature, but with a simpler transaction-based feature set. Precision shows the largest divergence from prior work, with the models presented here achieving around 0.6, while literature values sit near 0.9. Recall is where this work comes closest to the literature, with all values in the table falling within the 0.8 to 0.9 range. Finally, the Ponzi-class F1 shows a moderate gap, while literature models report values between 0.8 and 0.9, the models presented here achieve 0.69 and 0.70, respectively. Although this gap is recognisable, it reflects the cost of broader applicability. Transaction data is available for every deployed contract, whereas bytecode and source code access is limited to a small fraction of the Ethereum ecosystem.

A natural question arising from this comparison is whether the models are detecting behavioural patterns or exploiting the size difference between Ponzi and DApp contracts noted in the EDA. To address this, a sensitivity analysis was conducted to assess whether the classification results reflect a behavioural signal or are driven by contract size. To achieve this, only the three purely temporal features were used: *new_sender_rate*, *new_sender_decay*,

and *repeat_sender_ratio*. These features represent recruitment dynamics and investor return behaviour without relying on the size of a contract's transaction history. The models were trained using identical data splits and hyperparameters to those used for the full feature set. Random Forest achieved an AUC of 0.840 and a Macro F1 of 0.723, with XGBoost producing comparable results at AUC 0.830 and Macro F1 0.692. Although these results show a decrease compared to the complete four-feature models, they still demonstrate that temporal recruitment patterns possess independent discriminative power, regardless of contract scale.

Finally, SMOTE was evaluated as an alternative approach to class imbalance. Random Forest remained essentially unchanged, with Macro F1 moving from 0.814 to 0.820 and AUC dropping marginally from 0.925 to 0.921. XGBoost showed a similar pattern, with AUC dropping from 0.926 to 0.921 and Macro F1 from 0.816 to 0.807. Since SMOTE produced no meaningful benefit and class imbalance is already addressed by the models' built-in weighting mechanisms, this technique was ultimately not adopted. The full results are presented in Appendix A.

5 CONCLUSIONS AND FUTURE WORK

This thesis investigated whether Smart Ponzi schemes on Ethereum can be identified using transaction graph features alone, without access to contract source code or bytecode. The hypothesis was that Ponzi Schemes on the Ethereum network exhibit structural and temporal behavioural patterns in their ETH inflows and outflows transaction networks that distinguish them from legitimate Dapps, and that these patterns alone are sufficient to train competitive classifiers.

Four features were used in the final models. These features were constructed using the aggregated transaction networks of 1190 contracts after preprocessing. The four classifiers that were trained were Logistic Regression, Random Forest, XGBoost and SVM. The random forest and XGBoost models emerged as the two best performing models, achieving an AUC of 0.925 and 0.926 and Macro F1 scores of 0.814 and 0.816, respectively, on a held-out test set.

A sensitivity analysis was performed to test a model using only temporal features. These models performed worse than the four-feature models but confirmed that a behavioural signal exists independently of contract scale and can achieve an AUC of 0.84 and a Macro F1 of 0.72. This discovery directly addresses the concern that the models could be exploiting size differences between the two labels rather than detecting behavioural patterns that set them apart. The feature *new_sender_decay* ranked third in both models (RF and XGBoost) in the feature importance plots and proved sensitive to removal in the ablation study, strengthening the claim of its discriminative power.

The dominant feature amongst the two main models was *in_out_ratio*. Its importance must be interpreted carefully, as it is partly due to a data collection limitation, the lack of ERC-20 token transfer coverage. This limitation artificially alters the *out_degree* of DApps. Despite this concern, the feature remains discriminating, since Ponzi contracts cluster around a ratio of approximately one, suggesting a pattern that reflects their redistribution mechanism.

The transaction-based approach comes at a cost of precision relative to code or mixed-based methods, yet it offers broader applicability due to not requiring any type of code from the contract. The models presented here can be applied to all contracts as long as the contracts have a transaction history.

Regarding limitations, this project's main issue is its 1000-transaction cap per direction. For large, long-running Ponzi schemes, a transaction cap may mask the collapse phase. Although this is a serious limitation, it can be mitigated by simply not imposing the 1000-transaction limit, which will require more computing power, or by sampling the transactions and collecting, for example, the first 500 and the last 500. Data collection also introduces other important limitations, such as the fact that all features are derived solely from ETH transactions. Currently, a large number of Ethereum transactions involve ERC-20 token

transfers, which aren't recorded in the table from which the data was recovered. Extending the data collection coverage to these tokens may benefit the dataset.

The ratio of non-fraudulent to fraudulent transactions also presents a limitation, as the severe label imbalance makes detection more challenging. To address this, more advanced and sophisticated balancing methods could be employed.

Regarding feature selection, although Mutual Information was used to make the process model-agnostic, a fixed k=5 per fold and an 80% voting threshold were employed to determine selection pragmatically. Using alternative thresholds and methods could yield different, potentially better results, and the feature selection options were not evaluated exhaustively.

A structural limitation of the approach is its inability to classify micro-schemes. Contracts with fewer than five unique senders were excluded during preprocessing because the graph-based features are undefined at that scale. While this filter is methodologically justified, it means the model cannot flag smaller Ponzi schemes.

Finally, regarding the ground truth set, although it was derived from prior academic work, it may only cover a small subset of Smart Ponzi Schemes on the Ethereum Network. The Ponzi activity is mainly concentrated in the earlier period of the set (2017 to 2019). This could be problematic because the model might not perform well on newer Ponzi schemes if their mechanisms have evolved. One way to address this issue is to use or develop an updated dataset that includes more recent and diverse Smart Ponzi Schemes.

Beyond addressing all identified limitations, a natural progression of this research is early detection. Since the feature set is currently inexpensive to compute, an important question is how few transactions are needed for these features to reliably stabilise, thereby enabling early identification of a contract in its lifecycle, possibly before any significant financial damage occurs. This approach builds directly on the current methodology and offers a more practical framing of the detection challenge than solely after-the-fact classification.

BIBLIOGRAPHICAL REFERENCES

Artzrouni, M. (2009). The mathematics of Ponzi schemes. *Mathematical Social Sciences*, *58*(2), 190–201. https://doi.org/10.1016/j.mathsocsci.2009.05.003

Bartoletti, M., Carta, S., Cimoli, T., & Saia, R. (2020). Dissecting Ponzi schemes on Ethereum: Identification, analysis, and impact. *Future Generation Computer Systems*, *102*, 259–277. https://doi.org/10.1016/j.future.2019.08.014

Buterin, V. (2014). *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform.* https://ethereum.org/content/whitepaper/whitepaper-pdf/Ethereum_Whitepaper_-_Buterin_2014.pdf

Chen, W., Li, X., Sui, Y., He, N., Wang, H., Wu, L., & Luo, X. (2021). SADPonzi: Detecting and Characterizing Ponzi Schemes in Ethereum Smart Contracts. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, *5*(2), 1–30. https://doi.org/10.1145/3460093

Chen, W., Zheng, Z., Cui, J., Ngai, E., Zheng, P., & Zhou, Y. (2018). Detecting Ponzi Schemes on Ethereum: Towards Healthier Blockchain Technology. *Proceedings of the 2018 World Wide Web Conference on World Wide Web - WWW '18*, 1409–1418. https://doi.org/10.1145/3178876.3186046

Chen, W., Zheng, Z., Ngai, E. C.-H., Zheng, P., & Zhou, Y. (2019). Exploiting Blockchain Data to Detect Smart Ponzi Schemes on Ethereum. *IEEE Access*, *7*, 37575–37586. https://doi.org/10.1109/ACCESS.2019.2905769

Ethereum in BigQuery: A Public Dataset for smart contract analytics. (n.d.). Google Cloud Blog. Retrieved June 20, 2026, from https://cloud.google.com/blog/products/data-analytics/ethereum-bigquery-public-dataset-smart-contract-analytics



Hu, T., Liu, X., Chen, T., Zhang, X., Huang, X., Niu, W., Lu, J., Zhou, K., & Liu, Y. (2021). Transaction-based classification and detection approach for Ethereum smart contract. *Information Processing & Management*, *58*(2), 102462. https://doi.org/10.1016/j.ipm.2020.102462

Jung, E., Le Tilly, M., Gehani, A., & Ge, Y. (2019). Data Mining-Based Ethereum Fraud Detection. *2019 IEEE International Conference on Blockchain (Blockchain)*, 266–273. https://doi.org/10.1109/Blockchain.2019.00042

Krishnan, L. P., Vakilinia, I., Reddivari, S., & Ahuja, S. (2023). Scams and Solutions in Cryptocurrencies—A Survey Analyzing Existing Machine Learning Models. *Information*, *14*(3), 171. https://doi.org/10.3390/info14030171

Liang, R., Chen, J., He, K., Wu, Y., Deng, G., Du, R., & Wu, C. (2024). PonziGuard: Detecting Ponzi Schemes on Ethereum with Contract Runtime Behavior Graph (CRBG). *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 1–12. https://doi.org/10.1145/3597503.3623318

Liang, R., Chen, J., Wu, C., He, K., Wu, Y., Sun, W., Du, R., Zhao, Q., & Liu, Y. (2025). Towards Effective Detection of Ponzi Schemes on Ethereum with Contract Runtime Behavior Graph. *ACM Transactions on Software Engineering and Methodology*, *34*(4), 1–32. https://doi.org/10.1145/3707458

Peng, Z., & Boyle, P. P. (2025). *Ponzi Schemes: A Review*. SSRN. https://doi.org/10.2139/ssrn.5019934

Ponzi Scheme | Investor.gov. (n.d.). Retrieved December 30, 2025, from https://www.investor.gov/protect-your-investments/fraud/types-fraud/ponzi-scheme

Reports show scammers cashing in on crypto craze. (2022, June 3). Federal Trade Commission. https://www.ftc.gov/news-events/data-visualizations/data-spotlight/2022/06/reports-show-scammers-cashing-crypto-craze

Vasek, M., & Moore, T. (2019). Analyzing the Bitcoin Ponzi Scheme Ecosystem. In A. Zohar, I. Eyal, V. Teague, J. Clark, A. Bracciali, F. Pintore, & M. Sala (Eds.), *Financial Cryptography and Data Security* (Vol. 10958, pp. 101–112). Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-662-58820-8_8

What is Golem? (GNT) | Kraken. (n.d.). Retrieved June 15, 2026, from https://www.kraken.com/learn/what-is-golem-gnt

Wood, D. G. (2014). *ETHEREUM: A SECURE DECENTRALISED GENERALISED TRANSACTION LEDGER*. https://ethereum.org/content/developers/tutorials/yellow-paper-evm/yellow-paper-berlin.pdf

Yu, S., Jin, J., Xie, Y., Shen, J., & Xuan, Q. (2021). *Ponzi Scheme Detection in EthereumTransaction Network* (arXiv:2104.08456). arXiv. https://doi.org/10.48550/arXiv.2104.08456

Zhang, Y., Yu, W., Li, Z., Raza, S., & Cao, H. (2022). Detecting Ethereum Ponzi Schemes Based on Improved LightGBM Algorithm. *IEEE Transactions on Computational Social Systems*, *9*(2), 624–637. https://doi.org/10.1109/TCSS.2021.3088145

Zheng, Z., Chen, W., Zhong, Z., Chen, Z., & Lu, Y. (2023). Securing the Ethereum from Smart Ponzi Schemes: Identification Using Static Features. *ACM Transactions on Software Engineering and Methodology*, *32*(5), 1–28. https://doi.org/10.1145/3571847

APPENDIX A

Table 6 - Model Comparison with SMOTE

Model	AUC (Baseline)	Macro F1 (Baseline)	AUC (SMOTE)	Macro F1 (SMOTE)
Random Forest	0.925	0.814	0.921	0.820
XGBoost	0.926	0.816	0.921	0.807

Data with Purpose.

